

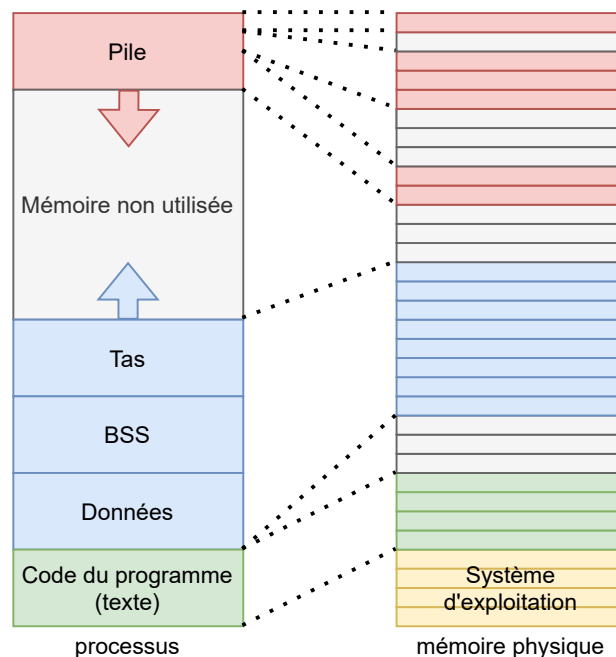
Devoir surveillé

Durée 1h30
(Corrections)

Exercice 1 (Cours - 4 pts)

1°) Donnez la représentation mémoire d'un processus en précisant chaque zone avec son rôle.

Solution : (2 pts) Dans le cours, la figure suivante a été présentée :



- Pile d'exécution : appels de fonctions avec variables
- Tas : données allouées dynamiquement
- Données non initialisées (zone appelée BSS)
- Données : variables globales initialisées ou variables statiques
- Code du programme

2°) Qu'est-ce qu'un inode ?

Solution : (1 pt) Un inode est présent dans les systèmes extX. C'est une structure qui permet de représenter un fichier ou un répertoire. Il possède un ensemble d'informations comme les attributs du fichier, le nombre de liens, etc. Il contient également une table d'adresses, chacune pointant vers un bloc du disque contenant les données du fichier/répertoire.

3°) Qu'est-ce que l'attente active et l'attente passive ? Donnez des exemples.

Solution : 1 pt Lorsqu'un processus attend en exécutant des instructions (une boucle infinie avec un sleep par exemple), c'est une attente active. Au contraire, avec une attente passive, le processus ne consomme pas de ressource (comme read sur un tube vide, par exemple).

Exercice 2 (Surveillance de processus - 5 pts)

Nous souhaitons mettre en place un système de surveillance de processus. Il est basé sur deux programmes : le *launcher* et le *watcher*. Le *launcher* démarre un programme et enregistre le processus correspondant auprès du *watcher*. Ce dernier attend l'enregistrement de processus et en même temps, il vérifie périodiquement que les processus enregistrés sont en cours d'exécution. Il affiche un message lorsqu'un processus s'est arrêté. L'enregistrement est réalisé à l'aide de signaux.

1°) Comment le *launcher* peut-il connaître le PID du processus démarré et comment peut-il l'envoyer au *watcher* ?

Solution : 1 pt Pour exécuter le programme, le *launcher* a besoin de créer un fils avec *fork* pour exécuter *execve*. Or, *fork* retourne le PID du fils. Le *launcher* peut l'envoyer à l'aide d'un signal temps réel avec *sigqueue*.

2°) Expliquez le fonctionnement complet du *launcher* et précisez tous les appels systèmes nécessaires. Il prend en arguments le nom du programme à exécuter et le PID du *watcher*.

Solution : 1 pt Le *launcher* commence par créer un processus fils avec *fork*. Il récupère ainsi son PID. Dans le processus fils, on exécute le programme à l'aide de *execve*. Une fois le fils créé, le programme envoie un signal au *watcher* à l'aide *sigqueue*, ce qui permet d'envoyer le PID du fils (et donc du programme).

Nous supposons que le *watcher* maintient la liste des processus à surveiller sous la forme d'un tableau.

3°) Comment le *watcher* peut-il vérifier que les processus surveillés sont toujours en cours d'exécution ? Précisez les appels systèmes.

Solution : 1 pt Il suffit d'envoyer un signal 0 vers le PID de chaque processus de la liste avec *kill*. Si le processus n'existe pas, cela renvoie -1 avec la variable *errno* qui prend la valeur *ESRCH*. Sinon, l'appel réussit et n'a pas d'effet.

4°) Donnez la partie du code permettant d'attendre le signal provenant du *launcher*. Cette attente doit être limitée à 0.5s.

Solution : 2 pts

```
struct timespec duration = { 0, 500000000 };
siginfo_t info;
sigset_t set;

sigfillset(&set);
sigprocmask(SIG_BLOCK, &set, NULL);

sigemptyset(&set);
sigaddset(&set, SIGRTMIN + 1);

sigtimedwait(&set, &info, &duration);
```

Exercice 3 (Traitements par flot - 6 pts)

Nous souhaitons mettre en place un flot de traitements sur des données réalisés par différents programmes indépendants (nommés *calculateur*). Chacun prend une série de données (pour simplifier, des entiers) saisie au clavier et écrit le résultat (une suite d'entiers) dans le terminal. Le résultat d'un *calculateur* peut être utilisé comme entrée pour un autre *calculateur*.

1°) Avec 3 *calculateurs* nommés *prog1*, *prog2* et *prog3*, donnez la ligne de commande dans le terminal pour réaliser le flot de traitement.

Solution : (0,5 pt) `./prog1 | ./prog2 | ./prog3`

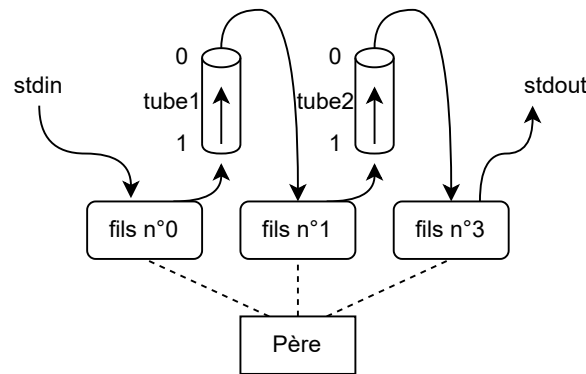
2°) Comment peut-on effectuer une redirection en C ? Précisez tous les appels systèmes.

Solution : (0,5 pt) On crée un tube nommé ou anonyme (*pipe* ou *mkfifo*) puis on redirige la sortie standard du premier programme vers l'entrée du tube puis on redirige l'entrée standard du deuxième programme vers la sortie du tube. Pour cela, on utilise la duplication de descripteurs avec *dup2* (ou *dup* et *close*).

Pour exécuter les *calculateurs*, nous écrivons un programme qui prend comme arguments les noms des programmes à exécuter. Il les exécute ensuite en redirigeant les entrées/sorties.

3°) Représentez l'architecture de l'application. Précisez les entrées/sorties.

Solution : (1 pt)



4°) Expliquez les différentes actions à réaliser pour ce programme. Précisez tous les appels systèmes.

Solution : (1 pt) On commence par créer les tubes ($n-1$ où n est le nombre de programmes) à l'aide de *pipe*. On crée ensuite un fils pour chaque programme avec *fork*. Dans le fils, on duplique l'entrée du tube (lecture = 0) sur l'entrée standard (*STDIN_FILENO*) et la sortie (écriture = 1) sur la sortie standard avec *dup2*. Il exécute ensuite le programme avec *execve*. On n'oublie pas de fermer tous les descripteurs des tubes qui ne servent à rien avec *close*. Le programme attend ensuite la fin des fils à l'aide de *wait*.

Nous souhaitons écrire la fonction `child(int in, int out, char *program)` où *in* est le descripteur de fichier correspondant à l'entrée du programme, *out* la sortie et *program* le nom du programme à exécuter.

5°) Expliquez pour chaque fils les valeurs à passer pour *in* et *out*.

Solution : (1 pt) Pour le premier fils, *in* est l'entrée standard (*STDIN_FILENO*) et *out* est l'entrée du premier tube (valeur 1). Pour le second, *in* est la sortie du premier tube (valeur 0) et *out* l'entrée du deuxième tube. Et ainsi de suite. Pour le dernier, *out* est la sortie standard (*STDOUT_FILENO*).

6°) Écrivez la fonction *child*.

Solution : (2 pts)

```
void child(int in, int out, char *program) {
    char *args[] = { program, NULL };

    if(in != STDIN_FILENO) {
        dup2(in, STDIN_FILENO);
        close(in);
    }

    if(out != STDOUT_FILENO) {
        dup2(out, STDOUT_FILENO);
        close(out);
    }

    execve(program, args, NULL);
}
```

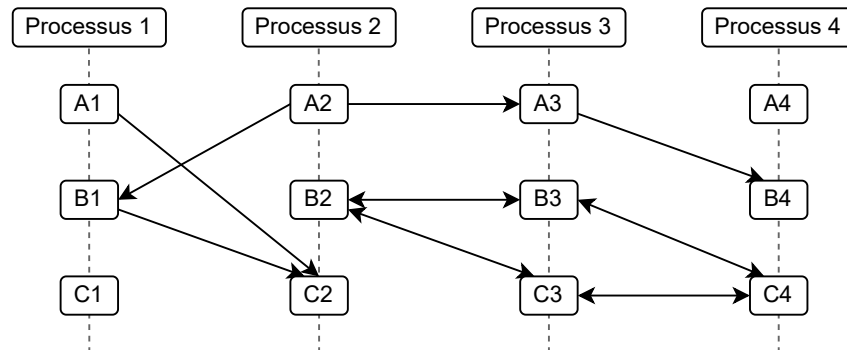
```

    exit(EXIT_FAILURE);
}

```

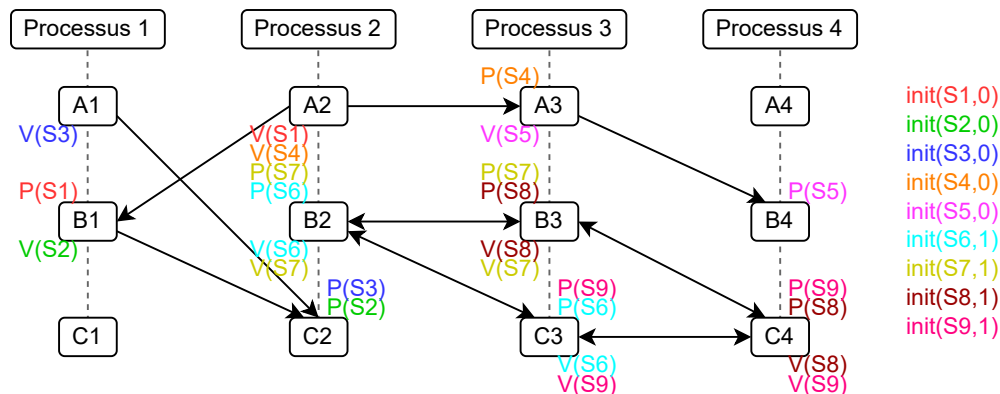
Exercice 4 (Sémaphores de Dijkstra et graphe d'allocation de ressources - 5 pts)

Soit le diagramme de précédence suivant, représentant 4 processus contenant chacun 3 blocs s'exécutant les uns après les autres :



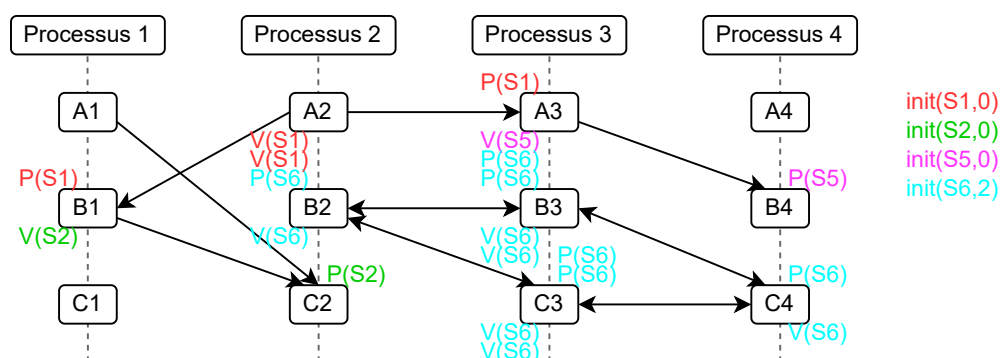
1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).

Solution : (1.5pts) Voici une solution :



2°) Proposez une solution pour réduire le nombre de sémaphores tout en conservant les contraintes du diagramme.

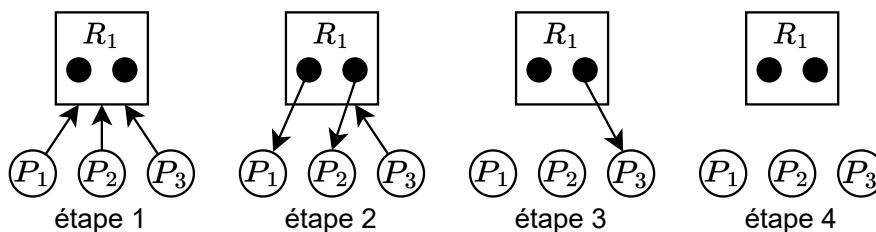
Solution : (1pt) Le sémaphore 3 peut être supprimé en réutilisant le sémaphore 2 : C2 attend 2 fois avec P(S2) et V(S2) après A1 et B1. Comme B1 s'exécute forcément après A1, on peut également supprimer simplement le sémaphore S3. Le sémaphore 4 peut être supprimé en réutilisant le sémaphore 1 : A2 fait 2 fois V(S1) après son exécution et on fait P(S1) avant A3 et B1. Comme B3 et C3 ne peuvent être exécutés en même temps, on peut réduire le nombre de sémaphores utilisés pour les 4 exclusions mutuelles.



Nous supposons une ressource R_1 comportant 2 exemplaires et 3 processus P_1 , P_2 et P_3 . Les trois processus demandent simultanément l'accès à un exemplaire de R_1 .

3°) Donnez toutes les étapes de l'exécution des demandes jusqu'à ce que tous les processus relâchent la ressource. Utilisez des graphes d'allocation de ressources différents.

Solution : (2pts) Voici une solution :



4°) D'une manière générale, est-il possible d'avoir un cas d'interblocage avec une ressource possédant deux exemplaires ?

Solution : (0.5pt) Oui, c'est possible. Si 2 processus possèdent chacun un exemplaire de la première ressource et sont en attente d'une deuxième ressource dont l'unique exemplaire est détenue par un autre processus qui lui-même attend un exemplaire de la première ressource.